

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 11_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 45.5

Section 1 : Coding

1. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3

2

0 1

1 0

0 0

1

Output: Maximum matching after removing vertex 1: 1

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int n, m;
```

```
int graph[15][15];
```

```
int matchR[15];
```

```
int visited[15];
```

```
int bpm(int u) {
```

```
    for (int v = 0; v < m; v++) {
```

```
        if (graph[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (matchR[v] == -1 || bpm(matchR[v])) {
```

```
                matchR[v] = u;
```

```
                return 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &m);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &graph[i][j]);
```

```
    int rem;
```

```
    scanf("%d", &rem);
```

```
    for (int j = 0; j < m; j++)
```

```
        graph[rem][j] = 0;
```

```
    for (int j = 0; j < m; j++)
```

```
        matchR[j] = -1;
```

```
    int result = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (i == rem) continue;
```

```
        memset(visited, 0, sizeof(visited));
```

```
        if (bpm(i))
```

```
            result++;
```

```
    }
```

```
    printf("Maximum matching after removing vertex %d: %d", rem, result);
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Charlie is working on optimizing the hiring process for interns at a tech startup. There are 4 open positions and 4 applicants, each having different skill sets.

Charlie wants to design a system that matches applicants to positions based on their suitability. The suitability is represented using a binary matrix, where:

1 indicates the applicant is suitable for the position. 0 indicates the applicant is not suitable for the position.

Each applicant can be assigned to only one position, and each position can accept only one applicant.

The goal is to determine the maximum number of positions that can be filled by assigning suitable applicants.

Input Format

The input consists of 4 lines.

Each line contains 4 space-separated integers (0 or 1).

Each row represents an applicant.

Each column represents a job position.

Output Format

The output prints the result in the following format: "The job can hire up to X applicants" where X represents the maximum number of applicants that can be assigned to positions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 1 0 0

1 0 0 0

0 0 1 1

0 0 0 1

Output: The job can hire up to 4 applicants

Answer

```
#include <stdio.h>
```

```

#include <string.h>

int graph[4][4];
int matchR[4];
int visited[4];

int bpm(int u) {
    for (int v = 0; v < 4; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;
            if (matchR[v] == -1 || bpm(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

int main() {
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < 4; i++)
        matchR[i] = -1;

    int result = 0;

    for (int i = 0; i < 4; i++) {
        memset(visited, 0, sizeof(visited));
        if (bpm(i))
            result++;
    }

    printf("The job can hire up to %d applicants", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Emily is working on a project that involves managing relationships between two sets of entities, set U and set V. She wants to determine the maximum matching in a bipartite graph to optimize connections without overlap.

Emily also needs the ability to remove a specific connection (edge) between two entities in order to analyze different scenarios and measure the impact on the size of the maximum matching.

Your task is to compute the size of the maximum matching before and after removing a given edge.

Note: Vertices in set U are numbered from 1 to n , and vertices in set V are numbered from 1 to m .

Input Format

The first line contains an integer n , representing the number of vertices in set U.

The second line contains an integer m , representing the number of vertices in set V.

The third line contains an integer e , representing the number of edges.

The next e lines each contain two integers u and v , representing an edge between vertex u (from set U) and vertex v (from set V).

The last line contains two integers u and v , representing the edge that must be removed.

Output Format

The first line displays: "Size of maximum matching before edge removal: X"

The second line displays: "Size of maximum matching after edge removal: Y"

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3

3

3

0 1

0 2

1 2

1 2

Output: Size of maximum matching before edge removal: 1

Size of maximum matching after edge removal: 0

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int n, m, e;
```

```
int graph[105][105];
```

```
int matchR[105];
```

```
int visited[105];
```

```
int bpm(int u) {
```

```
    for (int v = 0; v <= m; v++) {
```

```
        if (graph[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (matchR[v] == -1 || bpm(matchR[v])) {
```

```
                matchR[v] = u;
```

```
                return 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
int maxMatching() {
```

```
    memset(matchR, -1, sizeof(matchR));
```

```
    int count = 0;
```

```
    for (int u = 0; u <= n; u++) {
```

```
        memset(visited, 0, sizeof(visited));
```

```
        if (bpm(u))
```

```
            count++;
```

```
    }
```

```

    return count;
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    int before = maxMatching();

    int ru, rv;
    scanf("%d %d", &ru, &rv);
    graph[ru][rv] = 0;

    int after = maxMatching();

    printf("Size of maximum matching before edge removal: %d\n", before);
    printf("Size of maximum matching after edge removal: %d", after);

    return 0;
}

```

Status : Partially correct

Marks : 5.5/10

4. Problem Statement

In a computer lab, Alex is experimenting with task scheduling on a cluster of processors. The lab has 7 tasks that need to be executed and 5 processors available.

Each task can run only on certain processors due to compatibility constraints. These compatibilities are represented using a binary matrix,

where:

1 indicates the task can run on that processor. 0 indicates the task cannot run on that processor.

Each processor can execute only one task at a time, and each task can be assigned to only one processor. Alex wants to determine the maximum number of tasks that can be executed simultaneously.

Write a program to compute the maximum number of compatible task-processor assignments.

Input Format

The input consists of 7 lines.

Each line contains 5 space-separated integers (0 or 1).

Each row represents a task.

Each column represents a processor.

Output Format

The output prints the result in the format: "The processors can run a maximum of x tasks at one time.", where x is the maximum number of tasks that can be executed simultaneously.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 1 0 0 0

0 1 0 0 0

0 0 1 1 0

0 0 0 1 1

1 0 0 0 0

0 0 1 0 0

0 0 0 0 1

Output: The processors can run a maximum of 5 tasks at one time.

Answer

```
#include <stdio.h>
#include <string.h>
```

```
#define T 7
#define P 5
```

```
int graph[T][P];
int matchR[P];
int visited[P];
```

```
int bpm(int u) {
    for (int v = 0; v < P; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;
            if (matchR[v] == -1 || bpm(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}
```

```
int main() {
    for (int i = 0; i < T; i++)
        for (int j = 0; j < P; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < P; i++)
        matchR[i] = -1;
```

```
int result = 0;
```

```
for (int i = 0; i < T; i++) {
    memset(visited, 0, sizeof(visited));
    if (bpm(i))
        result++;
}
```

```
printf("The processors can run a maximum of %d tasks at one time.", result);
return 0;
```

}

Status : Correct

Marks : 10/10

5. Problem Statement

Alex is working on a project that involves pairing students from two different classes for a team project. The project's success depends on maximizing the number of compatible pairs between these two groups. Each student from one group can be paired with a student from the other group, but no student can belong to more than one pair.

Alex represents the compatibility between students using a binary adjacency matrix, where:

1 indicates that the students are compatible. 0 indicates that the students are not compatible.

Write a program to determine the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains two integers n and m , representing the number of students in the first group and second group, respectively.

The next n lines each contain m space-separated integers, representing the adjacency matrix of student compatibility.

Output Format

The output prints a single line in the following format: "Maximum Bipartite Matching: X", where X is the maximum number of compatible student pairs.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 3
1 1 0

```
1 0 1
0 1 0
1 0 0
```

Output: Maximum Bipartite Matching: 3

Answer

```
#include <stdio.h>
#include <string.h>
```

```
int n, m;
int graph[105][105];
int matchR[105];
int visited[105];

int bpm(int u) {
    for (int v = 0; v < m; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;
            if (matchR[v] == -1 || bpm(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}
```

```
int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < m; i++)
        matchR[i] = -1;

    int result = 0;

    for (int i = 0; i < n; i++) {
        memset(visited, 0, sizeof(visited));
        if (bpm(i))
```

```
    result++;  
}
```

```
printf("Maximum Bipartite Matching: %d", result);
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10